

Ứng dụng của phép nhân ma trận trong tin học

Yu Huacheng

Trường Trung học số 2 Hàng Châu, Chiết Giang

2008

Nguồn gốc: Applications of matrix multiplication in informatics

IOI China candidate team papers / 2008 / Day 1 / Yu Huacheng

Abstract

Ma trận được dùng rộng rãi trong toán học, và cũng có nhiều ứng dụng trong tin học. Bài viết này trình bày một số ứng dụng của phép nhân ma trận trên ba phương diện: tối ưu quy hoạch động, phép nhân trên ma trận kề của đồ thị, và đệ quy chia đôi.

Từ khóa: lũy thừa nhanh; nhân ma trận; tối ưu quy hoạch động; ma trận kề của đồ thị; đệ quy chia đôi.

Contents

1 Kiến thức chuẩn bị	1
1.1 Ma trận	1
1.2 Phép nhân ma trận	1
1.3 Lũy thừa nhanh	2
2 Tối ưu quy hoạch động	2
2.1 Đếm cây khung, NOI 2007	2
2.1.1 Mô tả bài toán	2
2.1.2 Phân tích	2
2.2 Text Generator, SPOJ 1676	3
2.2.1 Mô tả bài toán	3
2.2.2 Phân tích	3
2.3 Nhận xét	4
3 Phép nhân trên ma trận kề của đồ thị	5
3.1 Cá sấu đầm lầy, ZJTSC 2005	5
3.1.1 Mô tả bài toán	5
3.1.2 Phân tích	6
3.1.3 Nhận xét	6
3.2 Cow Relays, USACO November 2007	6
3.2.1 Mô tả bài toán	6

3.2.2	Phân tích	7
3.3	Đường đi tối thiểu hóa cạnh lớn nhất	7
3.3.1	Mô tả	7
3.3.2	Phân tích	7
3.3.3	Một số mở rộng	8
4	Phép nhân ma trận và đệ quy chia đôi	8
4.1	Alien Language, TopCoder Algorithm SRM 377, Div 1, 1000	8
4.1.1	Mô tả bài toán	8
4.1.2	Phân tích	9
4.1.3	Đệ quy chia đôi	9
4.1.4	Phép nhân ma trận	10
4.1.5	Mở rộng	10
4.2	Dice Contest, CEPC 2003	11
4.2.1	Mô tả bài toán	11
4.2.2	Phân tích	11
4.2.3	Đệ quy chia đôi	11
4.2.4	Phép nhân ma trận	11
4.3	Nhận xét	12

1 Kiến thức chuẩn bị

1.1 Ma trận

Một ma trận có thể xem là một bảng số kích thước $n \times m$, trong đó n, m là các số nguyên dương. Ví dụ:

$$\begin{bmatrix} 0 & 1 & 2 \\ 1 & 2 & 3 \end{bmatrix}, \quad [1].$$

Thông thường ta dùng $A[i, j]$ để chỉ phần tử ở hàng i , cột j .

1.2 Phép nhân ma trận

Cho hai ma trận A, B . Ta có thể nhân A với B khi và chỉ khi kích thước của A là $a \times b$, kích thước của B là $b \times c$, với a, b, c đều là số nguyên dương. Nếu $C = AB$, thì C có kích thước $a \times c$ và

$$C[i, j] = \sum_{k=1}^b A[i, k]B[k, j], \quad 1 \leq i \leq a, 1 \leq j \leq c.$$

Phép nhân ma trận thỏa luật kết hợp, tức là $(AB)C = A(BC)$. Giả sử kích thước của A, B, C lần lượt là $a \times b, b \times c, c \times d$. Khi đó

$$\begin{aligned} ((AB)C)[i,j] &= \sum_{l=1}^c \left(\sum_{k=1}^b A[i,k]B[k,l] \right) C[l,j] \\ &= \sum_{k=1}^b \sum_{l=1}^c A[i,k]B[k,l]C[l,j] \\ &= \sum_{k=1}^b A[i,k] \left(\sum_{l=1}^c B[k,l]C[l,j] \right) \\ &= (A(BC))[i,j]. \end{aligned}$$

Hai ma trận $(AB)C$ và $A(BC)$ đều có kích thước $a \times d$, nên đẳng thức được chứng minh.

Nếu tính trực tiếp theo định nghĩa, nhân một ma trận $a \times b$ với một ma trận $b \times c$ cần thời gian $\mathcal{O}(abc)$. Với hai ma trận $N \times N$, thuật toán đơn giản có độ phức tạp $\mathcal{O}(N^3)$. Một số thuật toán khác có độ phức tạp thấp hơn, chẳng hạn thuật toán Strassen khoảng $\mathcal{O}(N^{2.81})$; trong các thuật toán lý thuyết, Coppersmith–Winograd đạt khoảng $\mathcal{O}(N^{2.36})$.

1.3 Lũy thừa nhanh

Khi tính a^t với t là số nguyên không âm, ta thường tính đệ quy $a^{\lfloor t/2 \rfloor}$, rồi dùng giá trị này để tính a^t trong thời gian $\mathcal{O}(1)$:

$$a^t = \begin{cases} (a^{t/2})^2, & t \text{ chẵn}, \\ (a^{\lfloor t/2 \rfloor})^2 \cdot a, & t \text{ lẻ}. \end{cases}$$

Vì vậy độ phức tạp của lũy thừa nhanh là $\mathcal{O}(\log t)$.

Đặt $M^t = \underbrace{M \cdot M \cdots M}_{t \text{ lần}}$. Do phép nhân ma trận thỏa luật kết hợp, ta cũng có thể dùng lũy thừa nhanh để tính M^t . Nếu M có kích thước $N \times N$, thuật toán đơn giản cần $\mathcal{O}(N^3 \log t)$. Trong phần lớn các thuật toán của bài viết này, ta đều sẽ dùng lũy thừa nhanh của ma trận.

2 Tối ưu quy hoạch động

2.1 Đếm cây khung, NOI 2007

2.1.1 Mô tả bài toán

Cho đồ thị vô hướng G có N đỉnh, $N \leq 10^{15}$. Các đỉnh được đánh số; giữa đỉnh i và đỉnh j có cạnh khi và chỉ khi $|i - j| \leq k$, với $2 \leq k \leq 5$. Cho N, k , hãy tính số cây khung của G .

2.1.2 Phân tích

Phương pháp tính định thức được nêu trong đề không tận dụng tính chất đặc biệt của đồ thị. Một cây khung của đồ thị là một đồ thị con không có chu trình và làm cho mọi đỉnh liên thông với nhau. Vì k nhỏ, mỗi đỉnh chỉ nối với rất ít đỉnh gần nó. Ta có thể dùng quy hoạch động nén trạng thái: để kiểm tra liên thông và chu trình, trạng thái lưu quan hệ liên thông của k đỉnh ngay trước đỉnh hiện tại, cùng với vị trí hiện tại. Khi chuyển trạng thái, ta liệt kê những cạnh nào giữa đỉnh hiện tại và k đỉnh trước đó được chọn, rồi kiểm tra có tạo chu trình hay có thể làm mất liên thông hay không. Chẳng hạn, nếu đỉnh xa nhất phía trước không liên thông với các đỉnh trước nữa, nó buộc phải nối với đỉnh hiện tại. Gọi l_i là số cấu hình liên thông của i đỉnh. Độ phức tạp giảm từ $\mathcal{O}(N^3)$ xuống $\mathcal{O}(l_k 2^k N)$.

Dù đã tận dụng khá đầy đủ tính chất của đồ thị, độ phức tạp này vẫn chưa chấp nhận được với giới hạn của đề. Quan sát thấy rằng nếu quan hệ liên thông của k đỉnh trước là như nhau, thì bất kể đỉnh hiện tại có chỉ số nào, ta đều chuyển sang đỉnh kế tiếp theo cùng một cách. Một công cụ tự động hóa việc chuyển này chính là ma trận.

Gọi $f[i][j]$ là số cách đạt tới trạng thái mà đỉnh hiện tại là i , còn trạng thái liên thông của k đỉnh trước là j . Nếu

$$f[i][j] = \sum_{t=1}^{l_k} a_{j,t} f[i-1][t],$$

thì mỗi $f[i][j]$ chỉ phụ thuộc tuyến tính vào tất cả các $f[i-1][t]$. Đặt các giá trị $f[i-1][t]$ vào một ma trận hàng M_{i-1} kích thước $1 \times l_k$. Định nghĩa

$$D = \begin{bmatrix} a_{1,1} & a_{2,1} & \cdots & a_{l_k,1} \\ a_{1,2} & a_{2,2} & \cdots & a_{l_k,2} \\ \vdots & \vdots & \ddots & \vdots \\ a_{1,l_k} & a_{2,l_k} & \cdots & a_{l_k,l_k} \end{bmatrix}.$$

Khi đó $M_i = M_{i-1}D$, vì theo định nghĩa phép nhân ma trận,

$$M_i[1,j] = \sum_{t=1}^{l_k} M_{i-1}[1,t]D[t,j].$$

Trước hết dùng tìm kiếm để tính các hệ số $a_{j,t}$ và trạng thái ban đầu M_k . Sau đó dùng lũy thừa nhanh của ma trận để tính các bước còn lại. Độ phức tạp trở thành $\mathcal{O}(l_k^3 \log N)$. Qua tìm kiếm thu được $l_k = 52$, nên độ phức tạp này hoàn toàn khả thi.

2.2 Text Generator, SPOJ 1676

2.2.1 Mô tả bài toán

Cho N từ, $1 \leq N \leq 10$, mỗi từ có độ dài không quá 6. Hỏi có bao nhiêu xâu độ dài L , chỉ gồm chữ cái in hoa, chứa ít nhất một từ đã cho. Đáp án lấy theo modulo 10007, với $1 \leq L \leq 10^6$.

2.2.2 Phân tích

Điều kiện “chứa ít nhất một từ” không dễ xử lý trực tiếp. Ta có thể tính tổng số xâu là 26^L , rồi trừ đi số xâu không chứa bất kỳ từ đã cho nào. Trong phần này, “hợp lệ” nghĩa là không chứa từ cấm nào.

Vì mọi từ có độ dài không quá 6, nếu $i-1$ chữ cái đầu đã tạo thành một xâu hợp lệ, thì khi kiểm tra i chữ cái đầu, ta chỉ cần nhìn sáu chữ cái cuối. Một quy hoạch động đơn giản là

$$f[i][a_1][a_2][a_3][a_4][a_5],$$

biểu thị số cách tiếp tục sinh xâu hợp lệ khi i chữ cái đầu đã hợp lệ và năm chữ cái cuối lần lượt là $a_1, \dots, a_5$. Khi đó

$$f[i][a_1][a_2][a_3][a_4][a_5] = \sum_{\substack{A \leq c \leq Z \\ \text{legal}(a_1 a_2 a_3 a_4 a_5 c)}} f[i+1][a_2][a_3][a_4][a_5][c],$$

trong đó $\text{legal}(s)$ nghĩa là mọi hậu tố của s đều không phải là một từ đã cho. Số trạng thái là $\mathcal{O}(M^5 L)$, và ngay cả nếu tiền xử lý hàm legal , thời gian vẫn là $\mathcal{O}(M^6 L)$, với M là kích thước bảng chữ cái. Điều này không khả thi.

Vấn đề của thuật toán trên là có quá nhiều trạng thái lặp. Ví dụ nếu không có từ nào bắt đầu bằng $Aa_2a_3a_4a_5$ và cũng không có từ nào bắt đầu bằng $Ba_2a_3a_4a_5$, thì hai trạng thái tương ứng có công thức chuyển hoàn toàn giống nhau:

$$\begin{aligned} f[i][A][a_2][a_3][a_4][a_5] &= \sum_{A \leq c \leq Z} f[i+1][a_2][a_3][a_4][a_5][c] \\ &= f[i][B][a_2][a_3][a_4][a_5]. \end{aligned}$$

Do đó ta không nên chỉ quan tâm các chữ cái cụ thể phía trước, mà nên quan tâm trạng thái hiện tại có khả năng sinh ra những từ nào.

Đổi trạng thái thành $f[i][j][k]$: hiện đã điền i chữ cái, k chữ cái cuối đúng bằng tiền tố độ dài k của từ thứ j , và với mọi $l > k$, l chữ cái cuối không phải tiền tố của bất kỳ từ nào. Khi thêm chữ cái kế tiếp, ta xét nó có khớp chữ cái tiếp theo của từ j hay không. Nếu không khớp, nó có thể khớp tiền tố của một từ khác; nếu không khớp tiền tố nào, ta tìm hậu tố dài nhất là tiền tố của một từ. Tóm lại, chuyển trạng thái là

$$f[i][j][k] = \sum_{A \leq c \leq Z} f[i+1][\text{maxSuffix}(j, k, c)_0][\text{maxSuffix}(j, k, c)_1],$$

trong đó $\text{maxSuffix}(j, k, c)_0$ và $\text{maxSuffix}(j, k, c)_1$ lần lượt chỉ từ và độ dài của hậu tố dài nhất, sau khi lấy tiền tố độ dài k của từ j rồi thêm chữ cái c . Nếu có nhiều lựa chọn, lấy từ đầu tiên.

Số trạng thái là $\mathcal{O}(L \sum_i \text{len}_i)$. Nếu tìm hậu tố trực tiếp khi chuyển, độ phức tạp vẫn có thể rất lớn; phần tìm hậu tố có thể tiền xử lý. Sau khi tiền xử lý, độ phức tạp có dạng

$$\mathcal{O}\left(MN \sum_i \text{len}_i^3 + LM \sum_i \text{len}_i\right).$$

Trong trường hợp xấu nhất, lượng tính toán vẫn quá lớn, nhất là vì còn phải lấy modulo nhiều lần.

Điểm mấu chốt vẫn là: mọi $f[i]$ đều chuyển từ $f[i+1]$, và với các i khác nhau, cách chuyển hoàn toàn giống nhau. Ta đánh số lại hai chiều sau của trạng thái thành một chiều. Nếu S là số trạng thái hợp lệ sau khi đánh số, ta có

$$g[i][j] = \sum_{k=1}^S a_{j,k} g[i+1][k].$$

Các hệ số $a_{j,k}$ được tiền xử lý. Định nghĩa

$$D = \begin{bmatrix} a_{1,1} & a_{2,1} & \cdots & a_{S,1} \\ a_{1,2} & a_{2,2} & \cdots & a_{S,2} \\ \vdots & \vdots & \ddots & \vdots \\ a_{1,S} & a_{2,S} & \cdots & a_{S,S} \end{bmatrix}.$$

Ma trận ban đầu I có kích thước $1 \times S$, mọi phần tử đều bằng 1, biểu thị các giá trị $g[L][j]$. Tính ID^L bằng lũy thừa nhanh, ta thu được $g[0][j]$; đáp án của số xâu không chứa từ cấm là $g[0][0]$, với trạng thái 0 là xâu rỗng. Độ phức tạp là

$$\mathcal{O}\left(MN \sum_i \text{len}_i^3 + S^3 \log L\right),$$

trong giới hạn đề chỉ còn dưới vài triệu phép toán ma trận, đủ nhanh.

2.3 Nhận xét

Qua hai ví dụ trên, ta thấy một quy hoạch động có thể tối ưu bằng phép nhân ma trận khi thỏa các điều kiện sau:

1. Trạng thái phải là một chiều hoặc hai chiều. Nếu trạng thái ban đầu có nhiều hơn hai chiều, có thể mã hóa nhiều chiều thành một chiều; số trạng thái không đổi.
2. Mỗi trạng thái $f[i][j]$ chỉ được phụ thuộc tuyến tính vào các $f[i-1][k]$.
3. Ngay cả khi hai điều kiện trên đúng, ma trận chưa chắc giúp giảm độ phức tạp. Nếu ma trận chuyển khác nhau không theo quy luật theo i , ta không thể dùng lũy thừa nhanh. Thông thường các ma trận chuyển phải giống nhau, hoặc ít nhất xuất hiện tuần hoàn.
4. Nếu chiều thứ nhất có kích thước N , chiều thứ hai có kích thước M , thì phương pháp ma trận cần $\mathcal{O}(M^3 \log N)$, còn chuyển trực tiếp thường không quá $\mathcal{O}(M^2 N)$, đôi khi còn nhỏ hơn. Vì vậy ma trận thường chỉ đáng dùng khi M nhỏ và N rất lớn.

3 Phép nhân trên ma trận kề của đồ thị

Ma trận kề biểu diễn duy nhất một đồ thị, đồng thời có nhiều tính chất thú vị. Trước hết xét trường hợp đơn giản nhất: đồ thị vô hướng không trọng số có N đỉnh. Nếu a nối với b , đặt $G[a, b] = G[b, a] = 1$, ngược lại bằng 0. Xét G^2 :

$$G^2[a, b] = \sum_{i=1}^N G[a, i]G[i, b].$$

Mỗi tích $G[a, i]G[i, b]$ bằng 1 khi và chỉ khi tồn tại cạnh $a \rightarrow i$ và cạnh $i \rightarrow b$. Do đó $G^2[a, b]$ chính là số đường đi từ a đến b qua đúng một đỉnh trung gian. Tương tự,

$$G^3[a, b] = \sum_{i=1}^N \sum_{j=1}^N G[a, i]G[i, j]G[j, b],$$

chính là số đường đi từ a đến b qua đúng hai đỉnh trung gian.

Tổng quát hơn, $G^k[a, b]$ bằng số đường đi độ dài k từ a tới b . Chứng minh bằng quy nạp: vì $G^k = G^{k-1}G$,

$$G^k[a, b] = \sum_{i=1}^N G^{k-1}[a, i]G[i, b].$$

Ở đây $G^{k-1}[a, i]$ là số đường đi độ dài $k-1$ từ a tới i , còn $G[i, b]$ cho biết có cạnh từ i tới b hay không. Tích của chúng đếm các đường đi độ dài k có đỉnh áp chót là i . Khi $k=0$, G^0 là ma trận đơn vị, và $G^0[a, a] = 1$ đúng với đường đi độ dài 0 từ a tới chính nó. Lập luận này không dùng tính vô hướng, nên cũng đúng cho đồ thị có hướng.

Với đồ thị có cạnh song song, chỉ cần đặt $G[a, b]$ là số cạnh từ a tới b . Với vòng tự nối trong đồ thị có hướng, không cần xử lý đặc biệt. Với vòng tự nối trong đồ thị vô hướng, thường cộng 2 vào $G[a, a]$ để cạnh đó được xem như hai cạnh khác nhau khi tính G^k ; nếu chỉ cộng 1, tính chất tổng mọi phần tử của ma trận bằng hai lần số cạnh sẽ không còn đúng. Trong bài cụ thể cần chọn cách diễn giải phù hợp.

3.1 Cá sấu đầm lầy, ZJTSC 2005

3.1.1 Mô tả bài toán

Trong một ao có N trụ đá, một số cặp trụ được nối bằng cầu đá. Có N_{Fish} con cá ăn thịt di chuyển tuần hoàn giữa các trụ. Nếu chu kỳ là T , thì tại thời điểm i và $i+T$, con cá ở cùng một trụ. Một người bắt đầu ở trụ s , cần tới trụ e tại thời điểm K . Mỗi đơn vị thời gian phải đi qua đúng một cầu đá, và ở mọi thời điểm không được đứng cùng trụ với cá. Cho tất cả các cầu đá, chu kỳ và quy luật di chuyển của mỗi con cá, hãy tính số tuyến đường hợp lệ khác nhau, modulo 10000.

$$1 \leq N \leq 50, \quad 1 \leq K \leq 2 \cdot 10^9, \quad 1 \leq N_{\text{Fish}} \leq 20.$$

Bảo đảm tại thời điểm ban đầu, s là vị trí có thể đứng.

3.1.2 Phân tích

Dễ thiết kế một quy hoạch động. Gọi $f[i][j]$ là số cách đi tiếp để đúng thời điểm K tới e , khi hiện ở thời điểm i và đang đứng trên trụ j . Khi đó

$$f[i][j] = \begin{cases} 0, & \text{nếu } \neg \text{Can}[i][j], \\ \sum_{1 \leq k \leq N, G[j][k]=1} f[i+1][k], & \text{nếu } \text{Can}[i][j], \end{cases}$$

với điều kiện cuối

$$f[K][e] = \begin{cases} 0, & \text{nếu } \neg \text{Can}[K][e], \\ 1, & \text{nếu } \text{Can}[K][e]. \end{cases}$$

Ở đây $\text{Can}[i][j]$ cho biết tại thời điểm i có thể đứng trên trụ j hay không, còn $G[i][j]$ cho biết hai trụ có nối cầu hay không. Đáp án là $f[0][s]$. Số trạng thái là $\mathcal{O}(NK)$, mỗi trạng thái chuyển trong $\mathcal{O}(N)$, nên tổng độ phức tạp là $\mathcal{O}(N^2K)$, quá lớn.

Nếu không có cá, mọi thời điểm và mọi trụ đều có thể đi qua, bài toán trở thành đếm số đường đi độ dài đúng K giữa hai đỉnh trên một đồ thị cố định. Bài toán đó giải được bằng ma trận trong $\mathcal{O}(N^3 \log K)$.

Với cá xuất hiện, đồ thị thay đổi theo thời gian. Gọi G_i là đồ thị sau khi xóa các cạnh liên quan tới những trụ không thể đi qua tại thời điểm i . Ma trận đáp án cuối cùng là

$$\prod_{i=1}^K G_i.$$

Ta cần nhân nhanh dãy ma trận này. Điều kiện quan trọng là chu kỳ của mỗi con cá nằm trong 2, 3, 4, mà bội chung nhỏ nhất của 2, 3, 4 là 12. Vì vậy cứ sau 12 đơn vị thời gian, đồ thị quay lại như cũ, tức là $G_i = G_{i+12}$. Suy ra

$$\prod_{i=1}^K G_i = \left(\prod_{i=1}^{12} G_i \right)^{\lfloor K/12 \rfloor} \left(\prod_{i=1}^{K \bmod 12} G_i \right).$$

Tiền xử lý tích của 12 ma trận đầu, dùng lũy thừa nhanh cho phần lặp, còn phần dư nhiều nhất 12 ma trận thì nhân trực tiếp. Độ phức tạp là $\mathcal{O}(N^3 \log K)$.

3.1.3 Nhận xét

Từ bài này rút ra kết luận: dù đồ thị thay đổi theo thời gian, muốn đếm số đường đi có số cạnh xác định từ một điểm tới một điểm khác, ta chỉ cần nhân trực tiếp các ma trận kề của đồ thị tại từng thời điểm để thu được ma trận đáp án.

3.2 Cow Relays, USACO November 2007

3.2.1 Mô tả bài toán

Cho đồ thị vô hướng có trọng số gồm M cạnh. Hãy tìm đường đi ngắn nhất từ đỉnh S tới đỉnh E đi qua đúng K cạnh.

$$2 \leq M \leq 100, \quad 2 \leq K \leq 1000000.$$

Bảo đảm mỗi đỉnh xuất hiện trong cạnh có bậc ít nhất 2.

3.2.2 Phân tích

Bài toán rất giống đếm đường đi đi qua đúng K cạnh, nên ta muốn tìm một phép nhân ma trận tương tự. Tuy nhiên điều kiện “ngắn nhất” không thể biểu diễn bằng cộng và nhân thông thường.

Trước hết xét quy hoạch động. Gọi $f[i][j]$ là khoảng cách ngắn nhất để đi qua đúng i cạnh và tới đỉnh j . Khi đó

$$f[i][j] = \min_{k=1}^N (f[i-1][k] + G[k, j]),$$

trong đó $G[k, j]$ là trọng số cạnh giữa k, j , hoặc $+\infty$ nếu không có cạnh. Độ phức tạp là $\mathcal{O}(N^2 K)$.

Công thức này rất giống công thức đếm đường đi

$$g[i][j] = \sum_{k=1}^N g[i-1][k] \cdot G[k, j],$$

chỉ khác là phép nhân đổi thành phép cộng, còn phép cộng đổi thành phép lấy nhỏ nhất. Vì vậy ta định nghĩa lại phép nhân ma trận:

$$C[i, j] = \min_{k=1}^b (A[i, k] + B[k, j]),$$

với A kích thước $a \times b$, B kích thước $b \times c$, và $C = AB$ kích thước $a \times c$. Phép nhân này vẫn thỏa luật kết hợp:

$$\begin{aligned} ((AB)C)[i, j] &= \min_{l=1}^c \left(\min_{k=1}^b (A[i, k] + B[k, l]) + C[l, j] \right) \\ &= \min_{k=1}^b \min_{l=1}^c (A[i, k] + B[k, l] + C[l, j]) \\ &= \min_{k=1}^b \left(A[i, k] + \min_{l=1}^c (B[k, l] + C[l, j]) \right) \\ &= (A(BC))[i, j]. \end{aligned}$$

Áp dụng phép nhân ma trận đã sửa, ta giải bài toán trong $\mathcal{O}(N^3 \log K)$. Đề không nêu trực tiếp giới hạn N , nhưng vì $M \leq 100$ và mỗi đỉnh liên quan có bậc ít nhất 2, số đỉnh xuất hiện không quá 100, nên thuật toán rất nhanh.

3.3 Đường đi tối thiểu hóa cạnh lớn nhất

3.3.1 Mô tả

Cho một đồ thị có hướng. Hãy tìm một đường đi giữa hai đỉnh, đi qua đúng K cạnh, sao cho cạnh lớn nhất trên đường đi là nhỏ nhất có thể.

3.3.2 Phân tích

Với các bài toán yêu cầu nhỏ nhất của lớn nhất hoặc lớn nhất của nhỏ nhất, ta thường có thể nhị phân đáp án rồi kiểm tra. Ở đây, nếu đang kiểm tra giá trị x , chỉ các cạnh có trọng số không quá x được phép nằm trên đường đi. Bài toán trở thành kiểm tra giữa hai đỉnh có tồn tại đường đi đúng K cạnh hay không, có thể giải bằng lũy thừa ma trận kề. Vì đáp án chắc chắn là trọng số của một cạnh nào đó, ta sắp xếp các cạnh rồi nhị phân trên danh sách này. Độ phức tạp là $\mathcal{O}(N^3 \log N \log K)$.

Có thể giảm về $\mathcal{O}(N^3 \log K)$. Gọi $f[i][j]$ là giá trị nhỏ nhất có thể của cạnh lớn nhất trên đường đi xuất phát từ đỉnh đầu, đi đúng i cạnh và tới j . Khi đó

$$f[i][j] = \min_{k=1}^N \max\{f[i-1][k], G[k, j]\}.$$

Đối chiếu với công thức đếm đường đi, ta định nghĩa phép nhân ma trận:

$$(AB)[i, j] = \min_{k=1}^N \max\{A[i, k], B[k, j]\}.$$

Do min và max thỏa tính chất

$$\max \left\{ \min_i \max\{a_i, b_i\}, c \right\} = \min_i \max\{a_i, b_i, c\},$$

ta có chứng minh luật kết hợp tương tự:

$$\begin{aligned} ((AB)C)[i, j] &= \min_{l=1}^c \max \left\{ \min_{k=1}^b \max\{A[i, k], B[k, l]\}, C[l, j] \right\} \\ &= \min_{k=1}^b \min_{l=1}^c \max\{A[i, k], B[k, l], C[l, j]\} \\ &= \min_{k=1}^b \max \left\{ A[i, k], \min_{l=1}^c \max\{B[k, l], C[l, j]\} \right\} \\ &= (A(BC))[i, j]. \end{aligned}$$

Vì vậy thời gian là $\mathcal{O}(N^3 \log K)$.

3.3.3 Một số mở rộng

Mở rộng 1: cạnh nhỏ nhất là nhỏ nhất. Cho một đồ thị có hướng, hãy tìm đường đi đúng K cạnh sao cho cạnh nhỏ nhất trên đường đi là nhỏ nhất có thể. Bài này tương tự bài trên; chỉ cần đổi cả phép cộng và phép nhân trong phép nhân ma trận thành phép min. Tính phân phối được chứng minh gần giống như trên.

Mở rộng 2: không quá K cạnh. Cho một đồ thị có hướng, hãy tìm đường đi có số cạnh không quá K , sao cho cạnh lớn nhất trên đường đi là nhỏ nhất có thể. Cách tiện lợi là thêm vào mỗi đỉnh i một cạnh tự nối $i \rightarrow i$ có trọng số $-\infty$, rồi áp dụng thuật toán trên. Độ phức tạp không đổi.

Mở rộng 3: số cạnh trong đoạn $[K_1, K_2]$. Ta không thể lấy đáp án của $[0, K_2]$ trừ đi đáp án của $[0, K_1 - 1]$, vì phép lấy nhỏ nhất không có nghịch đảo. Nhìn lại mở rộng 2: thêm cạnh tự nối trọng số $-\infty$ tương đương với đổi mọi phần tử đường chéo chính của ma trận thành $-\infty$. Theo định nghĩa phép nhân đã sửa,

$$(AG)[i, j] = \min_{k=1}^M \max\{A[i, k], G[k, j]\}.$$

Khi $k = j$, ta có $\max\{A[i, k], G[k, j]\} = A[i, j]$, tức là giữ lại đáp án cũ; các hạng khác tạo ra đáp án mới. Gọi ma trận gốc là G_0 . Ta nhân trước K_1 lần G_0 , sau đó nhân $K_2 - K_1$ lần ma trận G có khả năng giữ đáp án. Đáp án là

$$G_0^{K_1} G^{K_2 - K_1}.$$

Dùng lũy thừa nhanh trực tiếp, độ phức tạp là $\mathcal{O}(N^3 \log K_2)$.

4 Phép nhân ma trận và đệ quy chia đôi

4.1 Alien Language, TopCoder Algorithm SRM 377, Div 1, 1000

4.1.1 Mô tả bài toán

Một bảng chữ cái gồm P nguyên âm và Q phụ âm. Mỗi từ có không quá N nguyên âm và không quá N phụ âm. Mỗi từ luôn là một đoạn nguyên âm nối tiếp một đoạn phụ âm, tức là mọi nguyên âm đứng

trước mọi phụ âm. Số nguyên âm và phụ âm đều có thể bằng 0, nhưng độ dài cả từ không được bằng 0. Một từ có thể mang trọng âm: không có trọng âm, có một trọng âm ở bất kỳ chữ cái nào, hoặc có hai trọng âm, một trên nguyên âm và một trên phụ âm. Hai từ viết giống nhau nhưng đặt trọng âm khác nhau được xem là khác nhau. Cho N, P, Q, M , với

$$1 \leq N, P, Q \leq 10^9, \quad 2 \leq M \leq 10^9,$$

hãy tính số từ khác nhau modulo M .

4.1.2 Phân tích

Bài này có nhiều tính toán đại số và dữ liệu lên tới 10^9 , nên ta muốn tìm một công thức trực tiếp. Nguyên âm và phụ âm không ảnh hưởng lẫn nhau, và hai phần có mô tả tương đương. Vì vậy chỉ cần giải bài toán con sau:

Cho một từ có không quá N chữ cái, mỗi chữ cái có K khả năng. Từ có thể không có trọng âm hoặc có đúng một trọng âm trên một chữ cái. Hãy tính số từ có thể có modulo M , ký hiệu là $S_{N,K}$.

Khi đó $S_{N,P}$ là số khả năng cho phần nguyên âm, $S_{N,Q}$ cho phần phụ âm. Loại bỏ trường hợp độ dài toàn bộ bằng 0, đáp án là

$$(S_{N,P}S_{N,Q} + M - 1) \bmod M.$$

Với độ dài i , có $(i + 1)K^i$ khả năng, vì có i vị trí đặt trọng âm hoặc không đặt trọng âm. Do đó

$$T = \sum_{i=0}^N (i + 1)K^i.$$

Nhân hai vế với K ,

$$KT = \sum_{i=1}^{N+1} iK^i.$$

Lấy hai công thức trừ nhau:

$$\begin{aligned} (K - 1)T &= (N + 1)K^{N+1} - \sum_{i=0}^N K^i \\ &= (N + 1)K^{N+1} - \frac{K^{N+1} - 1}{K - 1}, \end{aligned}$$

nên

$$T = \frac{(N + 1)K^{N+1}}{K - 1} - \frac{K^{N+1} - 1}{(K - 1)^2}.$$

Công thức này có phép chia, trong khi $K - 1$ chưa chắc nguyên tố cùng nhau với M . Ta cũng không thể tính số nguyên khổng lồ K^{N+1} rồi mới lấy modulo. Vì vậy phải quay lại công thức tổng không có phép chia.

4.1.3 Độ quy chia đôi

Biểu thức của $S_{N,K}$ có dạng bậc thang. Khi tách đôi, hai phần xấp xỉ trong hình của bản gốc giống hệt nhau, còn phần hình chữ nhật còn lại là một cấp số nhân quen thuộc. Phần xấp xỉ chính là $S_{N,K}$. Từ đó có công thức

$$\begin{aligned} S_{2N+1,K} &= \sum_{i=0}^{2N+1} (i + 1)K^i \\ &= (K^{N+1} + 1)S_{N,K} + K^{N+1}(N + 1) \sum_{i=0}^N K^i. \end{aligned}$$

Tổng hình học cũng có thể tính bằng chia đôi:

$$\sum_{i=0}^{2N+1} K^i = (K^{N+1} + 1) \sum_{i=0}^N K^i.$$

Các lũy thừa K^N đều tính được bằng lũy thừa nhanh. Nếu duy trì đồng thời $\sum_{i=0}^N K^i$ và K^N , thuật toán đạt $\mathcal{O}(\log N)$.

4.1.4 Phép nhân ma trận

Cách tính trực tiếp cần $\mathcal{O}(N)$, quá lớn. Ta dùng ma trận để tự động hóa truy hồi. Dù có thể duy trì $i+1$ và K^i , ta không thể nhân chúng trong trạng thái tuyến tính. Công thức

$$S_{N+1,K} \equiv K \cdot S_{N,K} + \sum_{i=0}^{N+1} K^i \pmod{M}$$

gợi ý một cách làm khác. Tổng phía sau được duy trì bởi

$$\sum_{i=0}^{N+1} K^i \equiv K \sum_{i=0}^N K^i + 1 \pmod{M}.$$

Đặt ma trận ban đầu

$$I = \begin{bmatrix} 1 & K+1 & 1 \end{bmatrix},$$

và ma trận chuyển

$$D = \begin{bmatrix} K & 0 & 0 \\ 1 & K & 0 \\ 0 & 1 & 1 \end{bmatrix}.$$

Trong quá trình truy hồi, ma trận hàng kích thước 1×3 lần lượt giữ

$$S_{t,K}, \quad \sum_{i=0}^{t+1} K^i, \quad 1.$$

Vì phép nhân ma trận chỉ dùng cộng và nhân, ta lấy modulo M ngay trong quá trình tính. Tính ID^N là giải xong bài toán, với độ phức tạp $\mathcal{O}(\log N)$.

4.1.5 Mở rộng

Bài toán trên tương đương tính $\sum_{i=1}^N ix^i$. Trong quá trình đó ta đã duy trì thêm $\sum_{i=1}^N x^i = \sum_{i=1}^N i^0 x^i$. Với số mũ lớn hơn, ta có thể dùng cách tương tự để tính

$$f_{N,a} = \sum_{i=1}^N i^a x^i.$$

Khi $a > 0$,

$$\begin{aligned} f_{N,a} - x f_{N-1,a} &= \sum_{k=1}^N (k^a - (k-1)^a) x^k \\ &= \sum_{k=1}^N \left(\sum_{j=0}^{a-1} k^j (-1)^{a-j+1} \binom{a}{j} \right) x^k \\ &= \sum_{j=0}^{a-1} \binom{a}{j} (-1)^{a-j+1} f_{N,j}. \end{aligned}$$

Do đó

$$f_{N,a} = xf_{N-1,a} + \sum_{j=0}^{a-1} \binom{a}{j} (-1)^{a-j+1} f_{N,j}.$$

Khi $a = 0$,

$$f_{N,0} = xf_{N-1,0} + x.$$

Vì $f_{N,i}$ phụ thuộc vào các $f_{N,j}$ với $j < i$, ta xử lý theo thứ tự tăng của a , biểu diễn lần lượt $f_{N,i}$ bằng các $f_{N-1,j}$ với $0 \leq j \leq i$. Trong ma trận hàng $1 \times (a+2)$, ta duy trì

$$f_{N,0}, f_{N,1}, \dots, f_{N,a}, 1.$$

Từ đó có thể tính đáp án bằng ma trận như bài gốc, với độ phức tạp $\mathcal{O}(a^3 \log N)$.

4.2 Dice Contest, CEPC 2003

4.2.1 Mô tả bài toán

Trên một bàn cờ rộng 4 ô và dài vô hạn có một con xúc xắc đặt tại (x_0, y_0) . Mỗi mặt của xúc xắc có một trọng số. Cần lăn xúc xắc tới (x_1, y_1) ; chi phí mỗi lần lăn là trọng số của mặt hướng lên sau khi lăn. Hãy tìm chi phí nhỏ nhất.

$$|x_0|, |x_1| \leq 10^9, \quad 1 \leq y_0, y_1 \leq 4,$$

và trọng số mỗi mặt là số nguyên dương không quá 50.

4.2.2 Phân tích

Giới hạn tọa độ rất lớn nên thuật toán đường đi ngắn nhất thông thường không đủ. Nhưng bàn cờ có cấu trúc đặc biệt: chỉ tọa độ ngang có phạm vi lớn, còn tọa độ dọc chỉ có 4 giá trị. Nếu $x_0 \neq 0$, ta tịnh tiến cả điểm đầu và điểm cuối để $x_0 = 0$. Nếu khi đó $x_1 < 0$, ta phản chiếu toàn bộ bàn cờ và xúc xắc. Vì vậy có thể giả sử $x_0 = 0$ và $x_1 \geq 0$.

4.2.3 Đệ quy chia đôi

Muốn tới một ô có $x = x_1$, đường đi phải đi qua cột $x = \lfloor x_1/2 \rfloor$. Chỉ cần tính khoảng cách ngắn nhất giữa mọi trạng thái ở cột $x = 0$ và mọi trạng thái ở cột giữa, cũng như từ cột giữa tới cột $x = x_1$. Nếu x_1 chẵn, do bàn cờ vô hạn theo chiều ngang, khoảng cách từ cột đầu tới cột giữa bằng khoảng cách tương ứng từ cột giữa tới cột cuối, nên chỉ cần tính một lần. Nếu x_1 lẻ, trước hết tính tới cột $x = x_1 - 1$, rồi dùng kết quả tiền xử lý cho khoảng cách tới cột $x = 1$.

Như vậy bài toán đệ quy về hai trường hợp $x = 0$ và $x = 1$. Hai trường hợp này có thể tiền xử lý bằng thuật toán đường đi ngắn nhất thông thường. Vì trọng số xúc xắc nhỏ, số trạng thái liên quan không nhiều, đồ thị lại thưa; SPFA hoặc Dijkstra dùng heap đều chạy nhanh. Gọi N là số trạng thái trong một cột. Mỗi tầng đệ quy cần $\mathcal{O}(N^3)$, có $\mathcal{O}(\log x_1)$ tầng, nên thời gian là $\mathcal{O}(N^3 \log x_1)$.

4.2.4 Phép nhân ma trận

Bài toán đường đi ngắn nhất giữa hai đỉnh với số cạnh cố định rất giống bài này. Xét một đường đi hoàn chỉnh; với mỗi i , $0 \leq i \leq x_1$, tồn tại ít nhất một trạng thái có $x = i$. Lấy trạng thái cuối cùng trên đường đi có $x = i$ làm trạng thái đại diện của cột i . Xem N trạng thái trong một cột là các đỉnh, thì việc đi từ trạng thái đại diện của cột i tới trạng thái đại diện của cột $i + 1$ giống như đi từ một đỉnh sang một đỉnh khác.

Ta chỉ cần tiền xử lý đường đi ngắn nhất giữa mọi cặp trạng thái của hai cột kề nhau, tức là các cạnh của đồ thị mới, rồi bài toán trở thành dạng đã giải: đường đi ngắn nhất qua đúng x_1 cạnh. Khi tiền xử lý, sau trạng thái đại diện của cột i , các trạng thái tiếp theo đều có $x > i$, nên chỉ cần tính các trạng thái có $x \geq 0$. Độ phức tạp cũng là $\mathcal{O}(N^3 \log x_1)$. Về mặt tiệm cận, nếu dùng thuật toán nhân ma trận nhanh hơn, có thể đạt độ phức tạp tốt hơn, chẳng hạn khoảng $\mathcal{O}(N^{2.36} \log x_1)$.

4.3 Nhận xét

Bản chất của lũy thừa nhanh ma trận chính là đệ quy chia đôi. Vì vậy trong nhiều trường hợp, hiệu quả của ma trận và đệ quy chia đôi là như nhau, thậm chí chương trình có thể rất giống nhau. Tuy nhiên ma trận vẫn có ưu thế trong nhiều tình huống. Chẳng hạn với mở rộng của bài Alien Language, nếu dùng đệ quy chia đôi thì rất khó suy ra công thức, đòi hỏi kỹ năng biến đổi đại số khá mạnh; còn với ma trận, chỉ cần khai triển nhị thức, chuyển về và thực hiện các biến đổi đại số cơ bản là có thể suy ra truy hồi. Ngoài ra, do đã có các thuật toán nhân ma trận tốt hơn $\mathcal{O}(N^3)$, đôi khi phương pháp ma trận còn có thể đạt hiệu quả cao hơn đệ quy chia đôi.

Tài liệu tham khảo

1. Wikipedia, <http://wikipedia.org/>.
2. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms, Second Edition*, 2001.

Lời cảm ơn

- Cảm ơn thầy Hu Xuhong đã hướng dẫn và giúp đỡ tác giả trong quá trình viết bài.
- Cảm ơn huấn luyện viên Liu Rujia đã hỗ trợ và cung cấp ví dụ Dice Contest.
- Cảm ơn những bạn học khác đã giúp đỡ.